Optimización básica de consultas

Bases de Datos 1 · Oracle 19c · Sesión de dos horas

La pregunta de hoy

- Hasta ahora escribíamos SQL para que devolviera lo correcto. Hoy nos preguntamos por qué a veces tarda, y qué podemos hacer al respecto.
- Todo lo que veamos se ejecuta en clase sobre una tabla de 300.000 filas. Los planes que van a ver son reales.

Las dos horas

- 0:00 Qué hace el motor con su consulta
- 0:15 El plan de ejecución · Demo 1
- 0:40 Índices: escaneo completo contra acceso por índice · Demo 2
- 1:10 Predicados que anulan un índice · Demo 3
- 1:35 Selectividad y estadísticas · Demo 4
- 1:50 Cinco reglas para llevarse

Qué hace el motor con su consulta

Usted dice qué, no cómo

- En SQL usted declara el resultado que quiere. El camino para obtenerlo lo escoge el motor, en un componente que se llama optimizador.
- Para una misma consulta suele haber varios caminos posibles: leer la tabla entera, entrar por un índice, unir en un orden o en otro.
- El optimizador les calcula un costo estimado a varios y se queda con el más barato. Ese costo es una unidad interna, no son segundos.

Entonces, ¿qué nos toca a nosotros?

- Darle buena información: estadísticas al día sobre el tamaño y el contenido de las tablas.
- Darle caminos: índices sobre las columnas por las que de verdad se filtra.
- No estorbarle: escribir condiciones que pueda aprovechar.
- **Optimizar no es escribir la consulta más bonita. Es que el motor tenga con qué elegir bien.**

El plan de ejecución

Es la explicación del motor, paso por paso

- Cada línea del plan es una operación: leer una tabla, recorrer un índice, unir dos conjuntos, ordenar, agrupar.
- Se lee por sangría: lo más indentado ocurre primero.

Las tres columnas que hay que mirar

- Operation: qué hace ese paso.
- Rows: cuántas filas ESTIMA el motor que va a devolver ese paso.
- Cost: el costo estimado. Sirve para comparar dos planes, no para prometer un tiempo.

Cómo se pide

```
EXPLAIN PLAN FOR  SELECT ... ;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(NULL,NULL,'BASIC +COST +ROWS +PREDICATE'));
```

  - En SQL Developer basta con F10 sobre la consulta.

- **Demo 1 — 01_demos.sql**

Cómo busca un índice

La idea, sin entrar en el detalle

- Un índice es una estructura aparte que guarda los valores de una columna ORDENADOS, y al lado la dirección exacta de la fila a la que pertenecen.
- Es el índice temático de un libro: los términos van en orden alfabético y cada uno dice en qué página está.
- **Buscar en él cuesta unos pocos saltos, sin importar si la tabla tiene mil filas o un millón. Esa es toda su gracia.**

De ahí salen las dos reglas del día

- Como está ordenado por el valor de la columna, sirve para buscar un valor, un rango o un comienzo de texto.
- Como guarda el valor tal como está en la tabla, cualquier cosa que transforme la columna en la consulta lo deja inservible.

Los dos accesos que hay que reconocer

TABLE ACCESS FULL

- Lee la tabla completa, bloque por bloque. Aparece cuando no hay índice aplicable, o cuando el motor decide que igual va a necesitar casi todo.

INDEX SCAN + TABLE ACCESS BY INDEX ROWID

- Busca los valores en el índice y luego va por las filas completas. Aparecen siempre juntos, uno debajo del otro.
- Si el filtro es por una llave primaria, el paso se llama INDEX UNIQUE SCAN: una sola fila posible.

El escaneo completo no es un error

- Cuando la consulta necesita buena parte de la tabla, leerla de corrido es más rápido que ir y volver miles de veces por el índice.
- **Por eso un índice puede hacer una consulta más lenta, y por eso el motor a veces lo ignora aunque exista.**
- **Demo 2 — 01_demos.sql**

Cómo anular un índice sin darse cuenta

Una función sobre la columna

```
WHERE UPPER(nombre) = 'GARCIA'      -- lee la tabla entera
WHERE nombre = 'GARCIA'             -- entra por el índice
```

- El índice guarda “Garcia”, no UPPER(“Garcia”). Ese valor no está en la estructura, así que el motor tiene que calcular la función fila por fila.

El mismo caso, disfrazado de fecha

```
WHERE TRUNC(fecha_venta) = DATE '2024-05-10'          -- no lo usa
WHERE fecha_venta >= DATE '2024-05-10'
  AND fecha_venta <  DATE '2024-05-11'                -- sí lo usa
```

- Es el error más común en producción, porque la columna guarda fecha y hora y uno solo quiere el día.
- **Fíjense en el “<” y no en BETWEEN: con BETWEEN hasta el 11 se cuelan las ventas de esa medianoche exacta.**

Otras dos formas de lograr lo mismo

Comparar tipos distintos

```
WHERE documento = 1004321        -- la columna es VARCHAR2
```

- Oracle no puede comparar texto con número, así que convierte la columna y en el plan aparece TO_NUMBER(“DOCUMENTO”): una función que nadie escribió.
- **Es el más difícil de ver, porque la consulta se lee perfectamente normal.**

```
WHERE documento = '1004321'      -- ahora sí
```

Comodín al principio

```
WHERE nombre LIKE 'GARCIA%'      -- entra por el índice
WHERE nombre LIKE '%GARCIA'      -- imposible
```

- Buscar lo que empieza por “GARCIA” es abrir el diccionario en la G. Buscar lo que termina en “GARCIA” obliga a leerlo entero: no hay orden que ayude.

▸ **Demo 3 — 01_demos.sql**

Cuándo un índice no sirve para nada

Selectividad

- Es qué tan pocas filas deja pasar un filtro. Filtrar por documento devuelve una fila; filtrar por canal, con dos valores posibles, devuelve media tabla.
- Un índice sirve para descartar. Si no descarta, no aporta: el motor lo ignora y hace bien.

Y además cuesta

- **Ocupa espacio, y hay que actualizarlo en cada INSERT, UPDATE y DELETE. Un índice que nadie usa es una deuda sin contrapartida.**
- La regla práctica: se indexan las columnas por las que se filtra a menudo y que tienen muchos valores distintos.
- **Demo 4 — 01_demos.sql**

Las estadísticas del optimizador

Qué son

- Un resumen que Oracle guarda de cada tabla: cuántas filas tiene, cuánto ocupa, cuántos valores distintos hay en cada columna.
- Con eso estima cuántas filas devolverá cada paso. Toda su decisión cuelga de esa estimación.

Qué pasa cuando están viejas

- La tabla creció a 300.000 filas pero el motor cree que tiene cien. Va a elegir un plan pensado para una tabla diminuta.
- **La consulta nunca estuvo mal: lo que estaba mal era lo que el motor creía saber.**

```
EXEC DBMS_STATS.GATHER_TABLE_STATS(USER,'VENTA',cascade=>TRUE);
```

- En una base normal esto corre solo, en la ventana de mantenimiento. El problema aparece cuando una carga masiva ocurre justo después.

Cinco reglas para llevarse

De esta clase al taller

- Ante una consulta lenta, lo primero es mirar el plan. Todo lo demás es adivinar.
- Un índice no es una mejora automática: ayuda cuando el filtro devuelve pocas filas, y estorba cuando devuelve muchas.
- No le aplique funciones a la columna en el WHERE. Reescriba la condición como un rango.
- Compare siempre el mismo tipo de dato, y desconfíe del comodín al principio de un LIKE.
- Si el plan no tiene sentido, sospeche de las estadísticas antes que del motor.

En el taller

- **Se permite la documentación oficial del motor. Lo que se pide no es la definición, es el porqué.**